

Relatório de Métricas

AutoJuri — Plataforma JurisFlow de Contestações

Repositório: GuilhermeADS13/API-JURISFLOW-CONTESTA-O

Branch: main · **Commit:** 236bc09

Data da análise: 2026-05-13

Ferramentas: pytest-cov, radon, vitest (v8)

Escopo: Backend (Python/FastAPI) + Frontend (React/Vite)

1. Cobertura de Código

1.1 Backend (Backend/App)

Comando: `pytest --cov=App --cov-report=term-missing --cov-report=html` Resultado: **220 testes passando** em 27.91s — **71% de cobertura global** (2.051 statements, 593 não cobertos).

Módulo	Stmts	Miss	Cobertura
<code>routes/contestacao.py</code>	39	0	100%
<code>routes/suporte.py</code>	20	0	100%
<code>models/n8n_response.py</code>	15	0	100%
<code>routes/feedback.py</code>	40	1	98%
<code>routes/usuario.py</code>	66	3	95%
<code>models/exemplar.py</code>	16	1	94%
<code>services/diff_minuta.py</code>	45	3	93%
<code>services/docx_editor.py</code>	69	4	94%
<code>models/feedback.py</code>	14	1	93%
<code>models/processo.py</code>	96	8	92%
<code>models/suporte.py</code>	58	6	90%
<code>routes/edicao.py</code>	79	10	87%
<code>models/edicao.py</code>	105	16	85%
<code>models/contestacao_por_peticao.py</code>	165	28	83%
<code>services/contestacao_docx_builder.py</code>	88	15	83%
<code>models/usuario.py</code>	107	20	81%

<code>routes/contestacao_peticao.py</code>	163	35	79%
<code>services/auth_service.py</code>	36	9	75%
<code>services/peticao_extractor.py</code>	178	49	72%
<code>limiter.py</code>	14	5	64%
<code>security.py</code>	122	58	52%
<code>database.py</code>	336	183	46%
<code>services/n8n_service.py</code>	105	81	23%
<code>services/suporte_email_service.py</code>	71	57	20%
TOTAL	2.051	593	71%

1.2 Frontend (Front comp/vite-project/src)

Comando: `npm run test:coverage` Resultado: **149 testes passando** em 8.34s — **84,10% statements / 91,79% branches / 95,65% funcs.**

Pasta / Arquivo	% Stmts	% Branch	% Funcs	% Lines
<code>utils/cases.js</code>	100	100	100	100
<code>utils/html.js</code>	100	100	100	100
<code>utils/validators.js</code>	100	100	100	100
<code>utils/files.js</code>	100	95,65	100	100
<code>utils/storage.js</code>	86,17	96,77	100	86,17
<code>lib/supabaseClient.js</code>	82,60	66,66	100	82,60
<code>components/AuthModal.jsx</code>	68,27	75	75	68,27

GLOBAL	84,10	91,79	95,65	84,10
--------	-------	-------	-------	-------

Observação: o threshold global do projeto está configurado em 90% (vite); o resultado atual fica 5,9 p.p. abaixo. Os módulos `App.jsx`, `MainPanelSection.jsx`, `RevisaoHumanaModal.jsx` e demais sections **não possuem testes** — não aparecem na tabela porque nenhum spec os importa, ficando fora do escopo da medição.

1.3 Análise da cobertura (não apenas o percentual)

Onde a cobertura está madura ($\geq 90\%$): - Toda a camada de **modelos Pydantic** (`processo`, `feedback`, `suporte`, `exemplar`, `n8n_response`) tem cobertura $\geq 90\%$ — validações de entrada estão garantidas. - **Rotas de domínio crítico do MVP** (`/contestacao`, `/suporte`, `/feedback`, `/usuario`) estão acima de 95%. - Serviços de geração e diff de minutas (`diff_minuta`, `docx_editor`) ficam $\geq 93\%$. - Camada utilitária do frontend (`validators`, `cases`, `html`, `files`) está em 100%.

Onde a cobertura é frágil (foco de risco):

Módulo	Cobertura	Risco associado
<code>services/suporte_email_service.py</code>	20%	Envio de e-mail SMTP não exercitado por teste — falha silenciosa em produção quando o setor de suporte é acionado.
<code>services/n8n_service.py</code>	23%	Cliente HTTP para o orquestrador n8n; cenários de erro (timeout, 5xx, schema inválido) não cobertos. Como o n8n dispara as chamadas Claude, falhas aqui param o produto.
<code>database.py</code>	46%	533 statements não cobertos cobrem caminhos de retry, mapeamento de status do dashboard e persistência pós-revisão.

<code>security.py</code>	52%	Funções de autenticação (<code>get_current_user</code> , helpers de hash legacy) com baixa cobertura — vetor de regressão silenciosa em segurança.
<code>AuthModal.jsx</code>	68%	Submissão real do formulário e fluxos de erro do Supabase pouco exercitados.

2. Complexidade Ciclomática (Radon — Backend)

Visão global: **168 blocos analisados, média A (4,32)** — saudável no agregado.

2.1 Pontos quentes (rank $\geq$ C, exigem atenção)

Função	Arquivo	CC	Rank
<code>contestar_por_peticao</code>	<code>routes/contestacao_peticao.py:67</code>	24	D
<code>montar_docx_com_modelo</code>	<code>services/contestacao_docx_builder.py:21</code>	21	D
<code>montar_docx_programa</code>	<code>services/contestacao_docx_builder.py:23</code>	23	C
<code>diff_secoes</code>	<code>services/diff_minuta.py:28</code>	14	C
<code>save_contestacao</code>	<code>database.py:572</code>	13	C
<code>editar_contestacao</code>	<code>routes/edicao.py:81</code>	13	C
<code>extrair_texto_peticao</code>	<code>services/peticao_extractor.py:95</code>	13	C
<code>_extrair_pdf</code>	<code>services/peticao_extractor.py:306</code>	13	C
<code>senha_forte</code>	<code>models/usuario.py:22</code>	11	C
<code>atualizar_minuta_editar</code>	<code>routes/contestacao_peticao.py:461</code>	11	C

<code>prefiltrar_secoes_juris</code>	<code>services/peticao_extractor.py:197</code>	11	C
--------------------------------------	--	----	---

Duas funções rank D ($CC \geq 21$) concentram lógica de orquestração — caminho `petição` → `extração` → `Claude` → `docx` em `contestar_por_peticao` e o montador de documento com modelo. São candidatos diretos a refatoração (`extract method`).

2.2 Índice de Manutenibilidade (MI)

Todos os arquivos retornam rank **A**, mas alguns merecem atenção pelo valor absoluto:

- `database.py` MI = **28,22** (mais baixo do projeto; arquivo de 978 linhas)
- `services/suporte_email_service.py` MI = **41,96**
- `models/usuario.py` MI = **43,98**
- `routes/contestacao_peticao.py` MI = **44,33**

3. LOC, Comentários e Duplicação

3.1 Tamanho

Métrica	Backend (<code>App/</code>)	Frontend (<code>src/</code> , sem testes)
Arquivos fonte	27	20
LOC total	4.614	4.127
SLOC (código)	3.278	—
LLOC (lógico)	2.346	—

Arquivos com tamanho acima do recomendado (>500 SLOC):

- `Backend/App/database.py` — **978 LOC** (módulo monolítico: conexão, sessões, usuários, contestações, dashboard, exemplares).
- `Backend/App/routes/contestacao_peticao.py` — **521 LOC** (acumula validação, upload, orquestração n8n e edição).
- `Front comp/vite-project/src/App.jsx` — **1.912 LOC** com **47 chamadas `useState`** em um único componente raiz (god component).

- `Front comp/vite-project/src/components/MainPanelSection.jsx` — **630 LOC**.

3.2 Densidade de comentários

Backend: **3% Comments/LOC, 11% (Comments+Multi)/LOC**. Densidade saudável para código autoexplicativo, mas baixa em módulos de regra de negócio densos (`database.py`, `n8n_service.py`) onde decisões de orquestração (retries, mapeamentos de status) se beneficiariam de comentários explicativos do *porquê*.

3.3 Duplicação e *code smells* observados

- **Broad `except Exception`**: em 6 arquivos (`security.py`, `peticao_extractor.py`, `n8n_service.py`, `database.py`, `contestacao_por_peticao.py`, `routes/contestacao_peticao.py`) — engole stack trace e dificulta diagnóstico em produção. Substituir por exceções específicas + log estruturado.
- **God component** em `src/App.jsx`: concentra autenticação, envio de casos, dashboard e suporte com 47 estados locais. Estado/lógica devem migrar para um `Context` ou hook customizado por domínio (`useAuth`, `useCases`, `useDashboard`).
- **Acoplamento de orquestração** em `contestar_por_peticao` (CC 24): mistura upload, validação MIME, chamada a `peticao_extractor`, persistência, fallback OCR e disparo n8n.
- **Configuração de URL de API duplicada** entre `App.jsx` e `config/api.js` (importa 6 constantes que poderiam ser um objeto único).

4. Interpretação dos resultados

Pontos fortes do projeto: 1. **Suíte de testes substancial e estável** — 220 backend + 149 frontend = **369 testes**, todos verdes; rodam em < 40s. 2. **Camada de modelos e utilitários muito bem coberta** ($\geq 90\%$), o que dá segurança para refatorar regras de validação sem regressão. 3. **Complexidade média baixa** (A 4,32) e **MI rank A** em todo o backend — o débito técnico está localizado em poucas funções, não espalhado. 4. **Convenção e estrutura clara** (`models/routes/services`), facilitando navegação e onboarding. 5. **Testes de segurança específicos** (`test_path_traversal`, `test_mime_validation`, `test_rate_limit`, `test_security_headers`) demonstram preocupação com OWASP.

Riscos do código atual: 1. **Camada de integração externa pouco testada** — `n8n_service` (23%) e `suporte_email_service` (20%) são *single points of failure* do produto e estão praticamente sem rede de proteção. 2. **`database.py` em 46%** com 978 linhas — qualquer refatoração nessa camada tem risco alto de regressão silenciosa em produção. 3. **`security.py` em 52%** — caminhos de autenticação não cobertos podem mascarar vulnerabilidades em produção, mesmo com testes de segurança no nível de rota. 4. **Frontend abaixo do threshold (84,1% < 90%)** porque componentes-chave (`App.jsx`, `MainPanelSection`, `RevisaoHumanaModal`) não têm specs — o pipeline `npm run test:coverage` falha

hoje no gate de qualidade. 5. **contestar_por_peticao (CC 24)** concentra o fluxo crítico do MVP; complexidade alta + cobertura de rota em 79% deixa branches importantes não exercitados.

Prioridades de melhoria (ordenadas por impacto/esforço):

1. **[ALTA]** Subir cobertura de `services/n8n_service.py` para $\geq 70\%$ com testes que mocam `httpx/requests` cobrindo timeout, 5xx e schema inválido — destrava confiabilidade do orquestrador.
2. **[ALTA]** Cobrir `services/suporte_email_service.py` (SMTP) ao menos nos caminhos felizes e nas três principais exceções (`SupportEmailConfigError`, `SupportEmailServiceError`, fallback).
3. **[ALTA]** Refatorar `contestar_por_peticao` em ≥ 3 funções menores (validação, extração, orquestração) — reduz CC para < 10 e melhora testabilidade.
4. **[MÉDIA]** Quebrar `database.py` em módulos por agregado (`db/usuario.py`, `db/sessao.py`, `db/contestacao.py`, `db/dashboard.py`) — sobe MI e cobertura natural.
5. **[MÉDIA]** Adicionar specs para `App.jsx/MainPanelSection.jsx` cobrindo o golden path (login → envio de petição → exibição no dashboard) para colocar o frontend acima dos 90% de threshold.
6. **[MÉDIA]** Substituir `except Exception:` genéricos por exceções específicas com log estruturado nos 6 arquivos identificados.
7. **[BAIXA]** Extrair estado de `App.jsx` para hooks por domínio (`useAuth`, `useCases`) — reduz acoplamento e abre caminho para testes unitários do componente raiz.

5. Reprodutibilidade

```
# Backend
cd Backend
pip install -r requirements-dev.txt pytest-cov radon
pytest --cov=App --cov-report=term-missing --cov-report=html:coverage_html
python -m radon cc App -a -s -nC # complexidade ciclomatica (rank >= C)
python -m radon mi App -s # indice de manutenibilidade
python -m radon raw App -s # LOC/SLOC/comentários

# Frontend
cd "Front comp/vite-project"
npm install
npm run test:coverage # vitest + v8
```

Relatórios HTML gerados localmente: [Backend/coverage_html/index.html](#) e [Front comp/vite-project/coverage/index.html](#).